Documentation SMP Project
Audio to MIDI

Peter Wienken, Aaron Michael Weis

March 8, 2026

Contents

1 Project Description **3**
1.1 Goal and Motivation 3
1.2 Software and Libraries 3

2 Software Architecture **5**
2.1 Signal Processing Pipeline 5
2.2 Object-Oriented Design of the Core Algorithm 5

3 Audio Import **8**
3.1 Loading Audio Data 8
3.2 Noise Gate 8

4 FFT Analyse **8**
4.1 FFT Algorithm 8
4.2 Windowing 9

5 FLUX Detection **10**
5.1 Calculation of Logarithmic Magnitude Spectrum 10
5.2 Calculation of Spectral Flux 10
5.3 Half-Wave Rectification 11

6 Onset Detection **12**
6.1 Adaptive Threshold and offset 13
6.2 Identification of Peaks (Peak Picking) 15
6.3 Practical Application 15

7 MIDI Export **16**
7.1 Library and Input Parameters 16
7.2 Output Configuration 16

8 Conclusion and Outlook **17**

9 References **17**

1 Project Description

1.1 Goal and Motivation

The primary motivation behind this project is to develop a tool capable of extracting the kick drum, snare drum, and hi-hat from an existing audio track or drum loop, and translating their rhythmic patterns into a standard MIDI format.
Often, musicians and producers encounter inspiring drum breaks or rhythms in existing songs but lack the individual MIDI notes to easily manipulate or recreate them. By extracting these rhythmic components and converting them into MIDI, users are empowered to isolate the groove, apply it to their own virtual drum samplers, alter the rhythm for new compositions, or simply generate an accurate rhythmic notation of the original performance.

1.2 Software and Libraries

To achieve the complex task of audio separation and analysis, a combination of machine learning models and digital signal processing libraries was employed. The software development for this project was conducted using **Visual Studio Code (VS Code)** as the primary Integrated Development Environment (IDE).
For the core algorithmic pipeline (Audio-to-MIDI translation), the following Python libraries were implemented:

- **Librosa:** Used for loading the audio files, preprocessing, and calculating the Short-Time Fourier Transform (STFT).
- **NumPy:** Essential for handling all multidimensional arrays and performing the mathematical calculations required for the spectral flux and adaptive peak picking.
- **Matplotlib:** Utilized for the visual evaluation of the algorithms, allowing the plotting of spectrograms, spectral flux curves, and the detected peak onsets.
- **Mido:** A library dedicated to MIDI handling. It translates the algorithmically determined temporal events (in seconds or frames) into standard MIDI messages (ticks, Note On/Off) and exports the multi-track `.mid` file.
- **Deep Learning Source Separation (PyTorch & TorchAudio):** To isolate the complete drum stem from a full, mixed song, a pre-trained neural network was utilized. Specifically, the project uses the `HDEMUCS_HIGH_MUSDB_PLUS` model provided by the `torchaudio.pipelines` module. This model separates the audio into discrete stems (vocals,

drums, bass, other) using PyTorch for tensor operations and GPU acceleration when available.

- **Individual Drum Extraction:** After isolating the overall drum stem, the tool **Rebeat** (`https://rebeatapp.com/`) was used to further separate this stem into the individual percussive components (kick, snare, and hi-hat).

2 Software Architecture

2.1 Signal Processing Pipeline

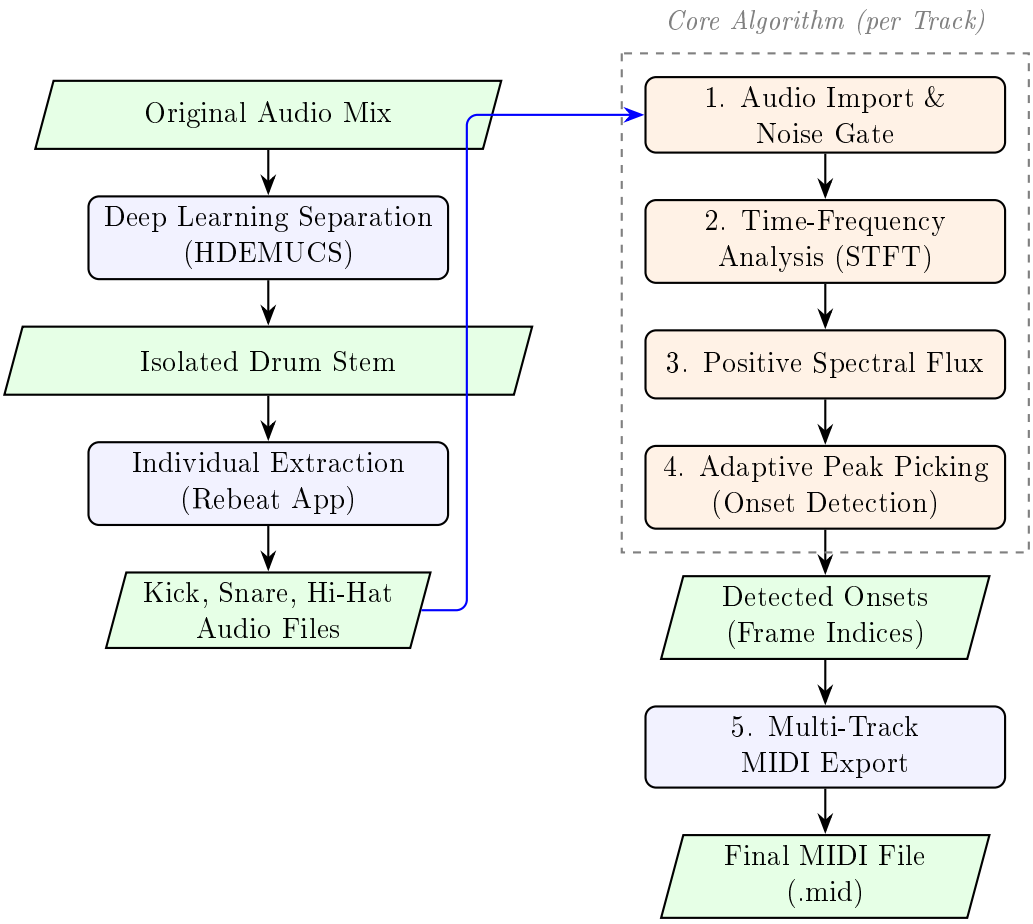

Figure 1: Flowchart of the complete Audio-to-MIDI Signal Pipeline.

2.2 Object-Oriented Design of the Core Algorithm

The project was implemented using a strict object-oriented programming (OOP) approach. Each phase of the audio analysis and processing is encapsulated within a specific class.
This approach ensures a strict separation of concerns. The respective data state, such as the current audio array, STFT calculations, or detected peaks, is stored as an attribute of the corresponding object. This avoids global variables and significantly simplifies parameter passing between the different processing steps.

The following section details the modular structure of the Python files and the role of each class:

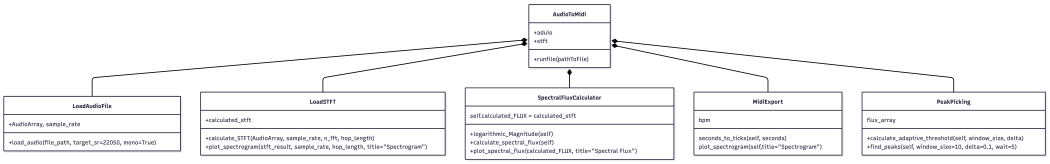

Figure 2: Class diagram of the Audio-to-MIDI object-oriented architecture.

`main.py`: This file serves as the central entry point of the application. Here, the main controller class `AudioToMidi` is instantiated, initialized with the file paths for the different drum components (kick, snare, hi-hat), and the automated processing pipeline is started.

`AudioToMidi.py`: This file contains the `AudioToMidi` class, which acts as the central orchestrator of the workflow. It creates instances of the other processing classes and controls the sequential execution for each instrument.

`LoadAudioFile.py`: The `LoadAudioFile` class encapsulates the reading of audio data using the `librosa` library. The resulting audio array and the sampling rate are stored directly within the object as its state. Additionally, the class contains custom methods for preprocessing steps, such as an implemented noise gate.

`LoadSTFT.py`: This class is responsible for calculating the Short-Time Fourier Transform (STFT). The class stores the calculated spectrogram as an attribute and provides methods for direct visualization.

`SpectralFluxCalculator.py`: Here, the spectral flux is calculated from the STFT data. The `SpectralFluxCalculator` class holds the STFT data as an attribute and provides methods for calculating the logarithmic magnitude as well as positive signal changes to detect high-energy transients (onsets) in the frequency spectrum.

`PeakPicking.py`: The logic for the actual beat detection is isolated within the `PeakPicking` class. It internally calculates an adaptive threshold using convolution (moving average) over the spectral flux and accounts for debouncing parameters. The finally detected onset frames are stored as an object attribute.

`Filtering.py`: The `Filtering` class manages various elliptical bandpass filters for the specific frequency ranges of the kick, snare, and hi-hat. It utilizes `scipy.signal` and keeps the filter states ready as instance variables for subsequent filtering processes.

`MidiExport.py`: The final step of the pipeline is encapsulated in the `MidiExport` class, which handles the conversion of the detected frames into a MIDI file using the `mido` library. It takes care of converting frames to seconds and then to MIDI ticks, generates individual tracks for each instrument, and bundles them on the standard MIDI drum channel.

3 Audio Import

The first step in the pipeline is loading and preprocessing the audio files.

3.1 Loading Audio Data

The Python library `librosa` is used for the actual loading of the audio data. The `load_audio` method reads the specified file, converts it to a mono signal by default, and uses a target sample rate of 44100 Hz.

3.2 Noise Gate

After the signal has been successfully loaded, noise reduction is applied to remove background noise in quiet passages. This is realized by the `noise_gate` method, which cuts off all values below a certain threshold and sets them to 0. The noise gate threshold is calculated dynamically using percentiles.

1. To obtain an envelope of the signal, the absolute values of the array are first computed: env = |self.audio_array|.

2. The threshold is then defined as the 90th percentile of this envelope using the `np.percentile(env, 90)` function. This means the threshold is set so that 90% of the signal's amplitude values fall below it, effectively isolating the loudest 10% of the audio (the actual drum hits).

3. Finally, using the `np.where` function, all signal components whose absolute magnitude is smaller than this calculated threshold are filtered out and overwritten with zero.

4 FFT Analyse

4.1 FFT Algorithm

The Fast Fourier Transform (FFT) algorithm is used for computing the Discrete Fourier Transform (DFT) of a sequence. The DFT of a sequence $x(n)$ of length N is defined as:

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-j2\pi kn/N} \tag{1}$$

For every frequency bin k, the DFT computes a complex number $X(k)$ that represents the amplitude and phase of the corresponding frequency component in the input signal. To calculate N coefficients, $O(N^2)$ operations are required. This is because each of the N output coefficients requires the computation of a sum containing N terms.

The FFT algorithm separates the DFT into smaller DFTs, by recursively dividing the input sequence into even and odd indexed elements.

$$X(k) = \sum_{n=0}^{N/2-1} x(2n)e^{-j2\pi k(2n)/N} + \sum_{n=0}^{N/2-1} x(2n+1)e^{-j2\pi k(2n+1)/N} \tag{2}$$

For better understanding, the even and odd indexed elements can be represented as, by using the substitution:

$$X_{even}(k) = \sum_{n=0}^{N/2-1} x(2n)e^{-j2\pi kn/(N/2)}.X_{odd}(k) = \sum_{n=0}^{N/2-1} x(2n+1)e^{-j2\pi kn/(N/2)}$$

(3)

to get the following expression for the DFT coefficients:

$$X(k) = X_{even}(k) + e^{-j2\pi k/N} X_{odd}(k) \tag{4}$$

Because of the periodicity properties of the complex exponential function, the second half of the spectrum can be derived directly from the same intermediate results. By substituting $k + N/2$ into the exponential term, the following relation is obtained:

$$e^{-j2\pi(k+N/2)/N} = e^{-j2\pi k/N}\, e^{-j2\pi(N/2)/N} \tag{5}$$

$$= e^{-j2\pi k/N}\, e^{-j\pi} \tag{6}$$

$$= -e^{-j2\pi k/N} \tag{7}$$

This sign change leads to a second expression for the Fourier coefficients:

$$X(k+N/2) = X_{even}(k) - e^{-j2\pi k/N} X_{odd}(k), X(k) = X_{even}(k) + e^{-j2\pi k/N} X_{odd}(k) \tag{8}$$

By recursively applying this decomposition, the DFT can be computed in $O(N \log N)$ time, which is a significant improvement over the naive $O(N^2)$ approach.

4.2 Windowing

To generate the spectrogram, the input signal is divided into overlapping frames, and a window function is applied to each frame to minimize spectral leakage. Common window functions include Hanning, Hamming, and Blackman windows.

The result is a 2D array where each column represents the magnitude spectrum of a frame, and each row corresponds to a specific frequency bin. Example of a 2D array representing frames and frequency bins:

$$\mathbf{L} = \begin{bmatrix} 1 & 4 & 1 & 6 \\ 3 & 2 & 2 & 1 \\ 1 & 5 & 3 & 6 \end{bmatrix}$$

This array can be visualized as a spectrogram, showing how the frequency content of the signal evolves over time.

5 FLUX Detection

5.1 Calculation of Logarithmic Magnitude Spectrum

The logarithmic magnitude spectrum is calculated using the formula:

$$L(k) = \log_{10}(|X(k)|) \tag{9}$$

It transforms the linear magnitude spectrum into a logarithmic scale, which is more aligned with human auditory perception.

5.2 Calculation of Spectral Flux

The spectral flux is calculated as the difference between the logarithmic magnitude spectra of consecutive frames:

$$F(n) = \sum_{k=0}^{N-1} L_n(k) - L_{n-1}(k) \tag{10}$$

For each frame n, the spectral flux $F(n)$ is computed by summing the squared differences of the logarithmic magnitude spectra $L_n(k)$ and $L_{n-1}(k)$ across all frequency bins k. The frames are typically stored in a 2D array where each column represents a frame and each row represents a frequency bin. The difference between consecutive frames can be represented as:

$$\Delta L(k) = L_n(k) - L_{n-1}(k) \tag{11}$$

Example of a 2D array representing frames and frequency bins:

$$\mathbf{L} = \begin{bmatrix} 1 & 4 & 1 & 6 \\ 3 & 2 & 2 & 1 \\ 1 & 5 & 3 & 6 \end{bmatrix}$$

The resulting difference between consecutive frames is:

$$\Delta\mathbf{L} = \begin{bmatrix} 4 & 1 & 6 \\ 2 & 2 & 1 \\ 5 & 3 & 6 \end{bmatrix} - \begin{bmatrix} 1 & 4 & 1 \\ 3 & 2 & 2 \\ 1 & 5 & 3 \end{bmatrix} = \begin{bmatrix} 3 & -3 & 5 \\ -1 & 0 & -1 \\ 4 & -2 & 3 \end{bmatrix}$$

Then, the spectral flux is computed by summing these differences across all frequency bins for each frame:

$$F(n) = \sum_k |\Delta L(k)| \tag{12}$$

$\mathbf{F} = \begin{bmatrix} |3| + |(-1)| + |4| & |(-3)| + |0| + |(-2)| & |5| + |(-1)| + |3| \end{bmatrix}^T = \begin{bmatrix} 8 & 5 & 9 \end{bmatrix}^T$

The Code to calculate the spectral flux from the differences is as follows:

```
flux = np.sum((calculated_FLUX[:, 1:] - calculated_FLUX
    [:, :-1]))
```

5.3 Half-Wave Rectification

Half-wave rectification is applied to the spectral flux to retain only positive values, which correspond to increases in spectral energy. This is done by setting all negative values to zero:

$$F_{rect}(n) = \max(0, F(n)) \tag{13}$$

This step ensures that only significant increases in spectral energy are considered for further analysis.

For example, given the spectral flux values:

$$\Delta L = \begin{bmatrix} -2 & 2 & -4 \\ 3 & 1 & 5 \\ -1 & 4 & -6 \end{bmatrix} = \begin{bmatrix} 0 & 2 & 0 \\ 3 & 1 & 5 \\ 0 & 4 & 0 \end{bmatrix}$$

And Summed up:

$$F = \begin{bmatrix} 0+3+0 & 2+1+4 & 0+5+0 \end{bmatrix}^T = \begin{bmatrix} 3 & 7 & 5 \end{bmatrix}^T$$

6 Onset Detection

Onset detection is an essential processing step within the pipeline. The goal of this procedure is to identify the exact timings (onsets) of musical events. In the case of drum detection, drum hits are identified, from a novelty function, which is the calculated positive spectral flux.
Consequently, a peak is only registered if three criteria are met simultaneously:

1. **Amplitude:** The signal must be loud enough and exceed the local adaptive threshold.

2. **Local Maximum:** The considered point must have a larger value than its direct temporal neighbors (the frame before and after).

3. **De-bouncing (Minimum Distance):** A single musical event can cause fluctuations over several frames. To avoid multiple detections of the same beat, a defined waiting time (`wait`) must be maintained between two detected peaks.

6.1 Adaptive Threshold and offset

An adaptive threshold follows the rough envelope of the signal. Typically, a moving average is calculated over a time window of a defined size for this purpose. The moving average serves to smooth the spectral flux and forms the basis of the adaptive threshold. Mathematically, the filter can be described as a discrete convolution of the input signal with a filter kernel.
Let $x[n]$ be the discrete-time input signal (the spectral flux at frame n) and N the window size (`window_size`). The output of the filter $y[n]$ is the unweighted mean of the N adjacent samples.
The centered calculation (corresponding to the `mode='same'` parameter in the code) can be expressed by the following sum formula:

$$y[n] = \frac{1}{N} \sum_{k=-\lfloor \frac{N}{2} \rfloor}^{\lfloor \frac{N-1}{2} \rfloor} x[n+k] \tag{14}$$

In signal processing, this operation is typically formulated as a discrete convolution. The signal $x[n]$ is convolved with the impulse response (the kernel) $h[n]$:

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k] \tag{15}$$

The filter kernel $h[n]$ is a rectangular window of length N, where all values are equally weighted. This corresponds to the code line `kernel = np.ones(window_size) / window_size`:

$$h[n] = \begin{cases} \frac{1}{N} & \text{for } 0 \leq n < N \\ 0 & \text{otherwise} \end{cases} \tag{16}$$

Through this convolution operation, short-term noise is suppressed and the fundamental dynamic envelope (the trend) of the signal is extracted, onto which the offset for the final threshold is subsequently added. To prevent background noise (noise floor) from being falsely detected as a peak, an offset (delta) is added to this average. This offset can be static or dynamically adapt to signal blocks.
In the present implementation, the moving average is realized by a mathematical convolution (`np.convolve`) of the flux with a uniformly distributed kernel. The offset is calculated and added block-wise:

```python
def calculate_adaptive_threshold(self, window_size):
    # Filter kernel for moving average
    kernel = np.ones(window_size) / window_size
    moving_avg = np.convolve(self.flux, kernel, mode='
        same')

```

```python
    # Calculate and add offset (delta)
    deltas = self.calculate_delta(window_size, 3)
    for i in range(len(moving_avg)):
        delta_index = int(i / window_size)
        if delta_index < len(deltas):
            moving_avg[i] += deltas[delta_index]

    self.threshold_curve = moving_avg
    return self.threshold_curve

def calculate_delta(self, window_size, multiplier):
    # Block-wise calculation of the noise offset
    len_flux = len(self.flux)
    delta_values = np.zeros(int(np.ceil(len_flux/window_size)))
    for i in range(len(delta_values)):
        start = i * window_size
        end = min((i + 1) * window_size, len_flux)
        delta_values[i] = np.mean(self.flux[start:end]) * multiplier
    return delta_values
```

6.2 Identification of Peaks (Peak Picking)

Once the threshold curve is available, the algorithm iterates through the signal. At each point, it checks whether the current value exceeds the threshold, whether it is the tip of a local hill, and whether the time distance to the last detected peak is large enough (de-bouncing).
This can be implemented very directly as a loop with conditional checks.

```
def find_peaks(self, window_size, wait):
    thresholds = self.calculate_adaptive_threshold(window_size)
    found_peaks = []
    last_peak_index = -wait  # Initialization for debouncing
    n_frames = len(self.flux)

    for i in range(1, n_frames - 1):
        current_val = self.flux[i]
        current_thresh = thresholds[i]

        # Condition 1: Above the threshold?
        if current_val > current_thresh:
            # Condition 2: Local maximum?
            if current_val > self.flux[i-1] and current_val > self.flux[i+1]:
                # Condition 3: Debouncing time expired?
                if (i - last_peak_index) > wait:
                    found_peaks.append(i)
                    last_peak_index = i

    self.onsets = np.array(found_peaks)
    return self.onsets
```

6.3 Practical Application

The parameters for window size (`window_size`) and waiting time (`wait`) depend heavily on the characteristics of the signal to be analyzed. For percussive, low-frequency instruments like a kick drum, different parameters are often chosen compared to high-frequency, fast instruments like a hi-hat.

7 MIDI Export

The final step in the pipeline is the translation of the detected onset times into a standard musical format (MIDI).

7.1 Library and Input Parameters

To construct and write the MIDI files, the Python library `mido` is used. Generating an accurate MIDI sequence requires translating the algorithmic time units (FFT frames) back into absolute real-time units (seconds) and eventually into MIDI ticks.
Therefore, the export function requires the following parameters from the preceding signal processing stages:

- **Onset Frames:** The arrays containing the identified frame indices for each instrument (`kick_frames`, `snare_frames`, `hihat_frames`).
- **Sample Rate and Hop Length:** The `sample_rate` of the original audio and the `hop_length` used during the FFT. These are mathematically essential to convert the frame indices into precise time stamps in seconds.
- **BPM (Beats Per Minute):** A defined tempo used to calculate the correct MIDI ticks per beat.

7.2 Output Configuration

The generated output is a multi-track MIDI file. To ensure compatibility with standard digital audio workstations (DAWs) and synthesizers, the export maps the data according to the General MIDI (GM) standard:

- **Routing:** All generated events are routed to MIDI Channel 10 (indexed internally as channel 9), which is the standard channel reserved for percussion and drums.
- **Track Separation and Note Mapping:** The exporter creates three separate tracks within the file. Each track represents one specific drum element and triggers a designated MIDI note number:
  - **Kick Drum:** Note 36
  - **Snare Drum:** Note 38
  - **Closed Hi-Hat:** Note 42

This structured output allows the user to simply import the final `.mid` file into any audio software and play it back using virtual drum samplers.

8 Conclusion and Outlook

In this project, a signal processing pipeline was developed to convert drum audio signals into MIDI events. The implemented system combines classical digital signal processing methods such as STFT analysis, spectral flux calculation, and adaptive peak picking in order to detect drum onsets and transfer them into MIDI format.
At the beginning, an attempt was made to separate the individual drum instruments using classical filter techniques based on their typical frequency ranges. However, this approach did not produce reliable results, since the frequency ranges of kick, snare, and hi-hat overlap strongly in real audio recordings. For this reason, an AI-based source separation method was chosen in order to first isolate the drum stem from the complete audio signal.
Nevertheless, this separation was still not perfect. Even after the AI-based processing, the resulting signals still contained noise and overlapping components, which negatively affected the onset detection. Therefore, a noise gate was introduced as an additional preprocessing step to suppress low-amplitude background components and make the actual drum hits more prominent.
Overall, the developed system is capable of generating usable MIDI drum patterns from audio material. At the same time, the results show that perfect separation and detection remain challenging. Future improvements could be achieved through more advanced separation models or more robust detection methods.

9 References